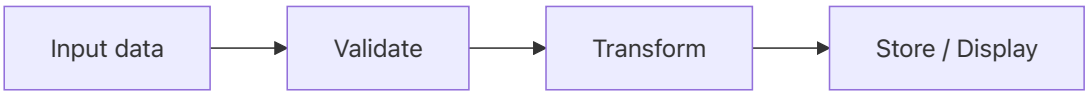
Python execution model

- Python is dynamically typed and interpreted, with most checks happening at runtime.
- Python code can be written in plain text files with the `.py` extension, not only in Jupyter notebooks (or Google Colab notebooks)
- Jupyter notebooks run code cell by cell; `.py` files commonly used to represent complete and reusable programs.
- A `.py` file can be run directly using the Python interpreter (e.g., `python script.py`).
- When a `.py` file is run, Python automatically compiles it before execution.
- The compilation step produces --bytecode--, a low-level representation of the program, which is further executed by Python Virtual Machine.

Sequential Logic

Data Flow



In [...]
```
print(2)
```

```
2
```

In [...]
```
# print(float(input()))
```

Input and Output

In [...]
```
h = 10
h = 'hello'
print(h)
```

```
2.0 + 2.0
```

```
hello
```

Out[…

```
4.0
```

In […

```
a = True
print(type(a))

a = 1
print(type(a))

# len(a)
# a + ' ' + '2.3'
# # int(a + 2.3)
```

```
<class 'bool'>
<class 'int'>
```

In […

```
a = 'suspicious'
len(a)
f'something {a}'
name = 'Debjit'
print('Hello Debjit')
# more on fstrings: https://realpython.com/python-f-s
print(f'Hello {name}')
type(a)
```

Out[…

```
10
```

Out[…

```
'something suspicious'
```

```
Hello Debjit
Hello Debjit
```

Out[…

```
str
```

Python Data Types

In Python, every value has a data type. Since Python is **dynamically typed**, you don't need to declare the type of a variable explicitly; the interpreter infers it at runtime.

In […

```
var1 = 2
var2 = 5.0
var3 = False
var4 = "Machine Learning"
```

```
print("Value of var1 var2:", var2, var1) #wrong!
print(f"Value of var1 : {var1}, var2: {var2}")
print("Value of var2 :", var2)
print("Value of var3 :", var3)
print("Value of var4 :", var4)

type(var1), type(var2), type(var3), type(var4)
```

```
Value of var1 var2: 5.0 2
Value of var1 : 2, var2: 5.0
Value of var2 : 5.0
Value of var3 : False
Value of var4 : Machine Learning
```

Out[…] (int, float, bool, str)

In […]
```
temp = 25
f"The current temperature is {temp}."
'2+3'
for i in range(5):
    print(f'guntur_{i+1}.jpg')
```

Out[…] 'The current temperature is 25.'

Out[…] '2+3'

```
guntur_1.jpg
guntur_2.jpg
guntur_3.jpg
guntur_4.jpg
guntur_5.jpg
```

Numeric Types

Python supports integers, floating-point numbers, and complex numbers.

In […]
```
for i in range(5):
    f'vij_aug2026_{i+1}.jpg'
```

Out[…] 'vij_aug2026_1.jpg'

Out[…] 'vij_aug2026_2.jpg'

Out[…] 'vij_aug2026_3.jpg'

Out[…] 'vij_aug2026_4.jpg'

Out[…] 'vij_aug2026_5.jpg'

In […]
```python
# Integer (int)
x = 10
print(f"x is {type(x)}")

# Float (float): Numbers with decimals
y = 10
print(f"y is {type(y)}")

# Complex (complex): written with a 'j' for the imagi
z = 3 + 5j
print(f"z is {type(z)}")
z.conjugate()
z.__abs__()
```

```
x is <class 'int'>
y is <class 'int'>
z is <class 'complex'>
```

Out[…] (3-5j)

Out[…] 5.830951894845301

In […]
```python
import cmath
cmath.polar(z)
```

Out[…] (5.830951894845301, 1.0303768265243125)

Sequence Types

Sequences allow you to store multiple items in an organized manner.

Strings (str)

Strings are sequences of Unicode characters wrapped in quotes.

In […]
```python
message = f"Hello, student!"
print(message)

print(f"First character: {message[0]}")    # Indexing
print(f"Last character: {message[-1]}")    # Indexing

# Slicing: access `n` characters from beginning and e
```

```
n = 1
print(f'{message[0:5]}')
print(f"First {n}: {message[:n]}")
print(f"Last {n}: {message[-n:]}")
message
# message[7:]
```

```
Hello, student!
First character: H
Last character: !
Hello
First 1: H
Last 1: !
```

Out[…

```
'Hello, student!'
```

In […

```
a = 'srm-ap'
a[4:]
```

Out[…

```
'ap'
```

Lists (list)

Lists are **mutable** (changeable) ordered collections that can hold mixed data types.

In […

```
lst = [1, 1.1, 'a']
lst[1:3]
# for i in lst:
#     print(i)
```

Out[…

```
[1.1, 'a']
```

In […

```
fruits = ["apple", "banana", "mango"]
# fruits.insert(1,'a') # inserting
fruits
# id(fruits) # location of the list

print(f'I like to eat {fruits[2]}.')
# type(fruits[1])
fruits[2][3]
fruits.append("orange")
fruits[2] = "blueberry"
print(f"Modified List: {fruits}")
```

Out[…] ['apple', 'banana', 'mango']

```
I like to eat mango.
```

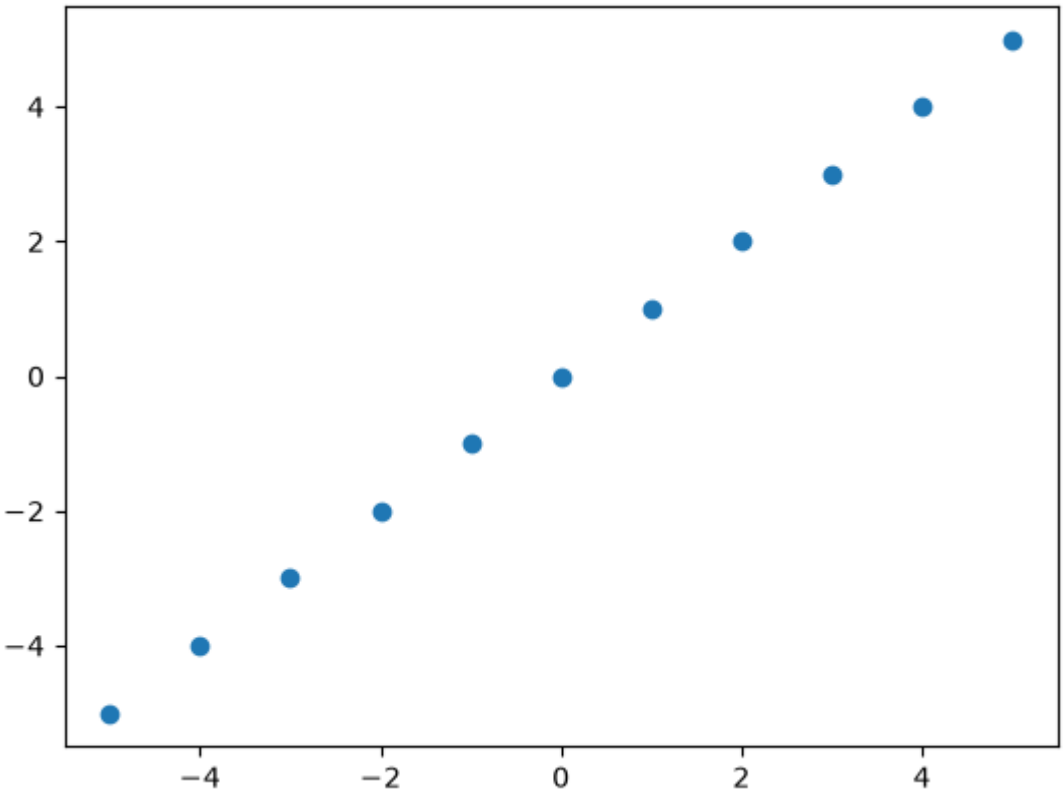
Out[…] 'g'

```
Modified List: ['apple', 'banana', 'blueberry', 'oran
ge']
```

In […]
```
# Example of how you can use a list!
xvals = [-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]
def parabola(x):
    return x
y = []
for x in xvals:
    y.append(parabola(x))
print(y)
import matplotlib.pyplot as plt
plt.scatter(xvals,y)
```

```
[-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]
```

Out[…] <matplotlib.collections.PathCollection at 0x12a5e260
0>



In […]
```
# lists can contain heterogeneous data types

items = [1, "abc", 3.5]
print(type(items[2]))
```

```
<class 'float'>
```

Tuples (tuple)

Tuples are **immutable** ordered collections. Once created, they cannot be changed.

In [...]
```python
coordinates = (15, 20, 30)
coordinates[0:2]
print(f"Tuple: {coordinates}")
# coordinates[0] = 15  # This would raise a TypeError
```

Out[...]
```
 (15, 20)
Tuple: (15, 20, 30)
```

Mapping Type: Dictionary (dict)

Dictionaries store data in **key–value pairs**. They are mutable and indexed by keys.

In [...]
```python
user = {
    "name": "Alice",
    "age": 25,
    "is_member": True
}
print(f"User Name: {user['name']}")

# user['weight']
x = {0: 'name', 1:
     'age'}

print(x[0], x[1])
```

```
User Name: Alice
name age
```

In [...]
```python
d = {5.5:'ab'}
d[5.5]
# d = [2]
# d[5]
```

Out[...]
```
 'ab'
```

Set Types (set)

A set is an unordered collection of **unique** items.

In […

```
numbers = {1, 'A', 'A', 'B', 2, 2, 3, 4, 4}
numbers
# print(f"Unique Set: {numbers}") # Removes duplicate
name = 'Arghya'
# name.lower()
print(f'{set(name.lower())}')

from collections import Counter
prt = 'AFDKSDJHFADJADFALKJDLFKJLD'
counter = Counter(prt)
counter
counter[0]
set(prt)
```

Out[…

```
{1, 2, 3, 4, 'A', 'B'}
{'g', 'a', 'y', 'r', 'h'}
```

Out[…

```
Counter({'D': 6, 'A': 4, 'F': 4, 'J': 4, 'K': 3, 'L'
: 3, 'S': 1, 'H': 1})
```

Out[…

```
0
```

Out[…

```
{'A', 'D', 'F', 'H', 'J', 'K', 'L', 'S'}
```

One complete example

In […

```
lst = [1,2,3,4,5]
lst
```

Out[…

```
[1, 2, 3, 4, 5]
```

In […

```
# --- LIST SLICING [start:stop:step] ---
nums = [0, 10, 20, 30, 40, 50]

sub = nums[1:4]       # [10, 20, 30] (index 1 to 3)
reverse = nums[::-1] # [50, 40, 30, 20, 10, 0]
every_other = nums[::2] # [0, 20, 40]
print(sub, reverse, every_other)

x = 'naans'
if x == x[::-1]:
    print('it is a palindrome')
else:
```

```
    print('not a palindrome')
```

```
[10, 20, 30] [50, 40, 30, 20, 10, 0] [0, 20, 40]
not a palindrome
```

In […

```
lst = []
for i in range(1,10,4):
    lst.append(i)
print(lst)
```

```
[1, 5, 9]
```

In […

```
# LIST COMPREHENSIONS: Pythonic way to create a list

normal_squares = []
for x in range(5):
    normal_squares.append(x**2)

print(normal_squares)
print(nums)
# Basic syntax: [expression for item in iterable if c
squares = [x**2 for x in range(5)]            # [0, 1,
evens = [x for x in nums if x % 2 == 0]      # [0, 20,

print(squares, evens)
```

```
[0, 1, 4, 9, 16]
[0, 10, 20, 30, 40, 50]
[0, 1, 4, 9, 16] [0, 10, 20, 30, 40, 50]
```

In […

```
# nums = list(range(50))
# evens = []
# for i in nums:
#     if i%2 ==0:
#         evens.append(i)
#     else:
#         continue
# evens
```

In […

```
items = ["apple", "banana"]
print(type(items))
items.append("cherry")   # Adds to end
popped = items.pop(1)    # Removes/returns "banana"
print(items, popped)
```

```
<class 'list'>
['apple', 'cherry'] banana
```